

```
// IN2011 Computer Networks
// Coursework 1 - Azure lab smoke test
//
// PURPOSE
// -----
// This program is a small example showing how your Node implementation can be
// used on the Azure virtual lab network.
//
// WHAT THIS FILE IS FOR
// -----
// - Starting a single CRN node on the Azure lab machine
// - Letting it wait for initial contact from other nodes
// - Performing a small number of simple read/write checks
// - Giving you something you can use while capturing Wireshark evidence
//
// WHAT THIS FILE IS NOT FOR
// -----
// This is NOT a complete test of the coursework.
// In particular, it does not fully test:
// - retransmission / timeout behaviour
// - duplicate or reordered packets
// - relay correctness
// - malformed message robustness
// - compare-and-swap edge cases
// - all message types and all topologies
//
// WORKFLOW
// -----
// A sensible workflow is:
// 1. develop and debug your Java code locally,
// 2. test multiple local nodes yourself,
// 3. run this file on the Azure lab machine,
// 4. capture Wireshark evidence there.
//
// USAGE
// ----
// Compile:
//   javac *.java
//
// Run:
//   java AzureLabTest your.email@city.ac.uk 10.x.x.x [port]
//
// Examples:
//   java AzureLabTest abc123@city.ac.uk 10.12.34.56
```

```

// java AzureLabTest abc123@city.ac.uk 10.12.34.56 20112
//
// NOTES
// ----
// - This should be run on the Azure lab machine, not your own laptop,
//   if you want to contact the provided lab nodes.
// - Choose a port in the recommended range if possible.
// - Your final submission must still follow the coursework rules exactly.

class AzureLabTest {

    private static final int DEFAULT_PORT = 20110;
    private static final int BOOTSTRAP_WAIT_MS = 12_000;

    public static void main(String[] args) {
        try {
            Config config = parseArgs(args);

            printBanner(config);

            // Create and initialise the node
            Node node = new Node();
            String nodeName = "N:" + config.emailAddress;
            node.setNodeName(nodeName);

            node.openPort(config.port);
            System.out.println("[OK] Node started");
            System.out.println("    node name : " + nodeName);
            System.out.println("    bind port : " + config.port);
            System.out.println();

            // Wait for initial contact from the Azure lab network
            System.out.println("[STEP 1] Waiting for initial contact from other nodes...");
            System.out.println("    This can take a little time on the Azure lab network.");
            node.handleIncomingMessages(BOOTSTRAP_WAIT_MS);
            System.out.println("[OK] Initial waiting stage finished");
            System.out.println();

            // Read known values that should already exist on the network
            System.out.println("[STEP 2] Reading known poem entries from the network...");
            int versesFound = 0;
            for (int i = 0; i < 7; i++) {
                String key = "D:jabberwocky" + i;
                String value = node.read(key);
            }
        }
    }
}

```

```

        if (value == null) {
            System.out.println("[WARN] Could not read key: " + key);
            System.out.println("    Possible reasons include:");
            System.out.println("        - your node has not yet learned enough peers");
            System.out.println("        - a message formatting/parsing problem");
            System.out.println("        - timeout / retransmission issues");
            System.out.println("        - the network test environment is not reachable");
        } else {
            versesFound++;
            System.out.println("[OK] " + key + " -> " + value);
        }
    }
}
System.out.println("[INFO] Poem entries found: " + versesFound + " / 7");
System.out.println();

// Write a simple marker value
System.out.println("[STEP 3] Writing a marker value to the network...");
String markerKey = "D:" + config.emailAddress;
String markerValue = "It works!";

boolean writeSuccess = node.write(markerKey, markerValue);
System.out.println("[INFO] write(" + markerKey + ") returned: " + writeSuccess);

String markerReadBack = node.read(markerKey);
if (markerReadBack == null) {
    System.out.println("[WARN] Marker value could not be read back.");
    System.out.println("    This suggests that write/read behaviour still needs
investigation.");
} else {
    System.out.println("[OK] Read-back succeeded: " + markerReadBack);
}
System.out.println();

// Advertise this node's address so other nodes can contact it
System.out.println("[STEP 4] Advertising this node's address...");
String addressValue = config.ipAddress + ":" + config.port;
boolean advertiseSuccess = node.write(nodeName, addressValue);
System.out.println("[INFO] write(" + nodeName + ", " + addressValue + ") returned: " +
advertiseSuccess);
System.out.println();

// Stay alive for inbound traffic and Wireshark capture
System.out.println("[STEP 5] Handling incoming messages...");

```

```

        System.out.println("    Leave this running while you inspect traffic / collect Wireshark
evidence.");
        System.out.println("    Press Ctrl+C when you want to stop.");
        node.handleIncomingMessages(0);

    } catch (IllegalArgumentException e) {
        System.err.println("Argument error: " + e.getMessage());
        printUsage();
        System.exit(1);
    } catch (Exception e) {
        System.err.println("Exception during AzureLabTest");
        e.printStackTrace(System.err);
        System.exit(2);
    }
}

private static Config parseArgs(String[] args) {
    if (args.length < 2 || args.length > 3) {
        throw new IllegalArgumentException("Expected 2 or 3 arguments.");
    }

    String emailAddress = args[0].trim();
    String ipAddress = args[1].trim();
    int port = DEFAULT_PORT;

    if (!emailAddress.contains("@")) {
        throw new IllegalArgumentException("First argument must be your email address.");
    }

    if (!looksLikeAzureIp(ipAddress)) {
        throw new IllegalArgumentException(
            "Second argument must be the Azure lab machine IP address (expected something
like 10.x.x.x).");
    }

    if (args.length == 3) {
        try {
            port = Integer.parseInt(args[2].trim());
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Third argument must be a valid integer port
number.");
        }
    }
}

```

```

        if (port < 1024 || port > 65535) {
            throw new IllegalArgumentException("Port must be between 1024 and 65535.");
        }
    }

    return new Config(emailAddress, ipAddress, port);
}

private static boolean looksLikeAzureIp(String ipAddress) {
    if (!ipAddress.startsWith("10.")) {
        return false;
    }

    String[] parts = ipAddress.split("\\.");
    if (parts.length != 4) {
        return false;
    }

    for (String part : parts) {
        try {
            int value = Integer.parseInt(part);
            if (value < 0 || value > 255) {
                return false;
            }
        } catch (NumberFormatException e) {
            return false;
        }
    }

    return true;
}

private static void printBanner(Config config) {
    System.out.println("=====");
    System.out.println(" AzureLabTest - CRN Azure smoke test");
    System.out.println("=====");
    System.out.println("Email    : " + config.emailAddress);
    System.out.println("Azure IP : " + config.ipAddress);
    System.out.println("Port     : " + config.port);
    System.out.println();
    System.out.println("Reminder:");
    System.out.println("- This is only a partial smoke test.");
    System.out.println("- It does NOT prove that all coursework features work.");
    System.out.println("- You should still do your own local and Azure-based testing.");
}

```

```

        System.out.println("=====");
        System.out.println();
    }

    private static void printUsage() {
        System.err.println();
        System.err.println("Usage:");
        System.err.println(" java AzureLabTest your.email@city.ac.uk 10.x.x.x [port]");
        System.err.println();
        System.err.println("Examples:");
        System.err.println(" java AzureLabTest abc123@city.ac.uk 10.12.34.56");
        System.err.println(" java AzureLabTest abc123@city.ac.uk 10.12.34.56 20112");
        System.err.println();
    }

    private static class Config {
        final String emailAddress;
        final String ipAddress;
        final int port;

        Config(String emailAddress, String ipAddress, int port) {
            this.emailAddress = emailAddress;
            this.ipAddress = ipAddress;
            this.port = port;
        }
    }
}

```